

✓ Construct a linked list containing 5 nodes and set the head to node 3

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

node1 = Node(10)
node2 = Node(20)
node3 = Node(30)
node4 = Node(40)
node5 = Node(50)

node1.next = node2
node2.next = node3
node3.next = node4
node4.next = node5

head = node3

print("Linked List:")
temp = node1
while temp:
    print(temp.data, end=" -> ")
    temp = temp.next
print("NULL")

print("Head =", head.data)
```

Linked List:
10 -> 20 -> 30 -> 40 -> 50 -> NULL
Head = 30

✓ Creating a Linked List in Python

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

node1 = Node(10)
node2 = Node(20)
```

```

node3 = Node(30)

node1.next = node2
node2.next = node3

head = node1

current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("NULL")

```

```
10 -> 20 -> 30 -> NULL
```

✓ Insertion of a Node (at Beginning)

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)

print("Original List:")
temp = head
while temp:
    print(temp.data, end=" -> ")
    temp = temp.next
print("NULL")

newNode = Node(5)
newNode.next = head
head = newNode

print("After Insertion:")
temp = head
while temp:
    print(temp.data, end=" -> ")
    temp = temp.next
print("NULL")

head.next = head.next.next

print("After Deletion:")
temp = head
while temp:

```

```

    print(temp.data, end=" -> ")
    temp = temp.next
print("NULL")

```

Original List:
 10 -> 20 -> 30 -> NULL
 After Insertion:
 5 -> 10 -> 20 -> 30 -> NULL
 After Deletion:
 5 -> 20 -> 30 -> NULL

✓ Insertion of a Node (at Ending)

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)

print("Original List:")
temp = head
while temp:
    print(temp.data, end=" -> ")
    temp = temp.next
print("NULL")

newNode = Node(40)

temp = head
while temp.next:
    temp = temp.next

temp.next = newNode

print("After Insertion at End:")
temp = head
while temp:
    print(temp.data, end=" -> ")
    temp = temp.next
print("NULL")

```

Original List:
 10 -> 20 -> 30 -> NULL
 After Insertion at End:
 10 -> 20 -> 30 -> 40 -> NULL

✓ Inserting at 3rd position

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)
head.next.next.next = Node(40)

newNode = Node(25)

temp = head
while temp.data != 20:
    temp = temp.next

newNode.next = temp.next
temp.next = newNode

temp = head
while temp:
    print(temp.data, end=" -> ")
    temp = temp.next
print("NULL")

10 -> 20 -> 25 -> 30 -> 40 -> NULL
```

✓ Create a linked list of any nodes and display the length of the linked list

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)
head.next.next.next = Node(40)
head.next.next.next.next = Node(50)
head.next.next.next.next.next = Node(60)
head.next.next.next.next.next.next = Node(70)
head.next.next.next.next.next.next.next = Node(80)
```

```
head.next.next.next.next.next.next.next.next = Node(90)
head.next.next.next.next.next.next.next.next.next.next = Node(100)

count = 0
temp = head

while temp:
    count += 1
    temp = temp.next

print("Length of Linked List =", count)
```

Length of Linked List = 10

✓ Create a Linked List

```
class linkedlist():
    def __init__(self, data):
        self.data = data
        self.next = None

n1 = linkedlist(10)
n2 = linkedlist(100)
n3 = linkedlist(1000)

n1.next = n2
n2.next = n3

head = n1

while head != None:
    print(head.data, end="->")
    head = head.next
print("None")
```

10->100->1000->None

✓ Sum of Values in Linked List

```
class linkedlist():
    def __init__(self, data):
        self.data = data
        self.next = None

n1 = linkedlist(10)
n2 = linkedlist(100)
```

```
n3 = linkedlist(1000)

n1.next = n2
n2.next = n3

head = n1

s = 0
while head != None:
    s += head.data
    head = head.next

print(s)
```

1110

✓ Product of Values in Linked List

```
class linkedlist():
    def __init__(self, data):
        self.data = data
        self.next = None

n1 = linkedlist(10)
n2 = linkedlist(100)
n3 = linkedlist(1000)

n1.next = n2
n2.next = n3

head = n1

prod = 1
while head != None:
    prod *= head.data
    head = head.next

print(prod)
```

1000000

✓ Count Number of Nodes

```
class linkedlist():
    def __init__(self, data):
        self.data = data
```

```
        self.next = None

n1 = linkedlist(10)
n2 = linkedlist(100)
n3 = linkedlist(1000)

n1.next = n2
n2.next = n3

head = n1

count = 0
while head != None:
    count += 1
    head = head.next

print(count)
```

3

✓ Search a Target in Linked List

```
class linkedlist():
    def __init__(self, data):
        self.data = data
        self.next = None

n1 = linkedlist(10)
n2 = linkedlist(100)
n3 = linkedlist(1000)

n1.next = n2
n2.next = n3

head = n1
target = int(input())

while head != None:
    if head.data == target:
        print("found")
        break
    head = head.next
else:
    print("not found")
```

10
found

Update a Node Value

```
class linkedlist():
    def __init__(self, data):
        self.data = data
        self.next = None

    def update(self, data):
        self.data = data

n1 = linkedlist(10)
n2 = linkedlist(100)
n3 = linkedlist(1000)

n2.update(99)

n1.next = n2
n2.next = n3

head = n1

while head != None:
    print(head.data, end="->")
    head = head.next
print("None")
```

10->99->1000->None

Insert After a Node

```
class linkedlist():
    def __init__(self, data):
        self.data = data
        self.next = None

    def insert(self, data):
        new_node = linkedlist(data)
        new_node.next = self.next
        self.next = new_node

n1 = linkedlist(10)
n2 = linkedlist(100)
n3 = linkedlist(1000)

n1.insert(99)
```



```

n1.next.next = n2
n2.next = n3

head = n1

while head != None:
    print(head.data, end="->")
    head = head.next
print("None")

```

10->99->100->1000->None

✓ Insert at Head

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def insert_at_head(head, data):
    new_node = Node(data)
    new_node.next = head
    return new_node

n1 = Node(10)
n3 = Node(40)
n4 = Node(5)

n1.next = n3
n3.next = n4

head = n1
head = insert_at_head(head, 99)

while head != None:
    print(head.data, end="->")
    head = head.next
print("None")

```

99->10->40->5->None

✓ Insert at Tail

```

class Node:
    def __init__(self, data):
        self.data = data

```

```

        self.next = None

def insert_at_tail(head, data):
    new_node = Node(data)

    if head == None:
        return new_node

    temp = head
    while temp.next != None:
        temp = temp.next

    temp.next = new_node
    return head

n1 = Node(10)
n3 = Node(40)
n4 = Node(5)

n1.next = n3
n3.next = n4

head = n1
head = insert_at_tail(head, 99)

while head != None:
    print(head.data, end="->")
    head = head.next
print("None")

```

10->40->5->99->None

✓ Insert at Specific Position

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def insert_at_head(head, data):
    new_node = Node(data)
    new_node.next = head
    return new_node

def insert_at_pos(head, data, pos):
    if pos == 0:
        return insert_at_head(head, data)

    new_node = Node(data)

```

```
temp = head

for i in range(pos - 1):
    temp = temp.next

new_node.next = temp.next
temp.next = new_node

return head
```

✓ Super Power ($a^b \bmod 1337$)

```
class Solution(object):
    def solve(self, a, b):
        return pow(a, int("".join(map(str, b))), 1337)

a = 2
b = [3]

obj = Solution()
print(obj.solve(a, b))
```

8

✓ Reverse a Linked List

```
class ListNode:
    def __init__(self, val):
        self.val = val
        self.next = None

class Solution:
    def reverseList(self, head):
        prev = None
        curr = head

        while curr:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt

        return prev

n1 = ListNode(10)
n2 = ListNode(20)
```

```
n3 = ListNode(30)

n1.next = n2
n2.next = n3

head = Solution().reverseList(n1)

while head:
    print(head.val, end=" ")
    head = head.next
```

30 20 10

```
class Solution(object):
    def reverseList(self, head):
        prev = None
        curr = head

        while curr != None:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt

        return prev
```